Week 7 Assignment

Repository Link:
https://github.com/Hughes-Tyler/AnalyticsCapstone

Report Link:
https://github.com/Hughes-Tyler/AnalyticsCapstone/blob/main/notebooks/main.ipynb

This week we trained our second modeling method, a Gradient Boosting classifier (scikit-learn's HistGradientBoostingClassifier), on the same processed startup dataset and the same train/validation split from Week 6. Like last week, we evaluated three hyperparameter variations that move from shallow to deep, calculated training and validation metrics for each, and selected the variation that generalizes best. We also compared the winner against our Week 6 Random Forest to see if boosting actually helped.

We chose Gradient Boosting mostly because of what we saw from Random Forest last week. Our target is heavily imbalanced (81.7% operating, 8.3% acquired, 8.0% closed, 2.0% IPO), and even our best forest (V2) lost most of its IPO recall going from training to validation (0.931 to 0.338). Random Forest trains each tree independently and then averages them, so the rare companies don't get any extra attention beyond the class weights. Gradient Boosting builds its trees one after another, and each new tree is fit to the mistakes the model is still making. That means the IPO, acquired, and closed companies that keep getting misclassified keep getting "pushed on" by later trees, which made sense for a problem where the whole point is finding the small classes. It also still works directly on our mix of 5 PCA components and 46 binary flags without any scaling, and it picks up interactions like funding amount together with country the same way the trees did in Week 6.

Gradient Boosting is definitely a more comlpicated model than Random Forest, both in training and in how many settings affect overfitting. For Random Forest, complexity mostly came down to how deep each tree could grow, and adding more trees just made the predictions more stable. Boosting works the other way around. It starts from shallow trees that underfit on their own, and every tree it adds lowers the bias a little more. So with boosting, more trees actually means a more flexible model, and training performance can keep climbing long after validation performance stops improving. The training is also

sequential, so the trees can't be built independently like a forest. Since we have four classes, the model builds one tree per class every round, so V1 ends up with 400 trees total, V2 with 800, and V3 with 1,200. The depth of each tree matters a lot too, since a depth 3 tree can have at most 8 leaves while a depth 8 tree can have up to 256.

We varied three hyperparameters together, moving from safe to aggressive so that V1 to V3 represents increasing capacity, the same way we set up Week 6. learning_rate controls how much each new tree is allowed to change the predictions. A small value learns slowly and cautiously, while a large value fits the training data quickly and overfits more easily. max_depth controls how complicated the patterns inside one tree can get. Deeper trees can pick up interactions between several features, but they can also start memorizing individual companies. max_iter is the number of boosting rounds, which is basically the same idea as n_estimators from Week 6. We chose these three because they are the settings that actually control overfitting in boosting, and they depend on each other. A low learning rate needs more rounds to reach the same fit, and deeper trees need fewer, so changing only one of them wouldn't really show the full range from underfitting to overfitting. We tested:

- **V1 - Shallow**: learning_rate=0.05, max_depth=3, max_iter=100. Small steps, simple trees, and few rounds, which we expected to underfit.
- **V2 - Moderate**: learning_rate=0.10, max_depth=6, max_iter=200. A middle ground we expected to balance fit and generalization.
- **V3 - Deep**: learning_rate=0.20, max_depth=8, max_iter=300. Big steps, deep trees, and many rounds, which we expected to overfit.

Every variation used class_weight='balanced' and random_state=42, same as Week 6. We also had to turn off two defaults so the variations would be a fair comparison. By default the model caps every tree at 31 leaves, which would quietly limit max_depth for V2 and V3, so we set max_leaf_nodes=None. It also turns on early stopping automatically for any dataset over 10,000 rows, which would hold out part of our training data and stop before the number of rounds we set, so we set early_stopping=False. We trained each variation on the 29,782 row training set, generated predictions on both that set and the 9,928 row validation set, and saved the models and a combined predictions file so the metrics could be calculated without retraining anything.

We kept the same metrics from Week 6 so the two models could be compared directly, and added one more. As mentioned in previous weeks, accuracy is a pretty bad grading measure here, since predicting "operating" for every company would score 81.7% while finding zero acquisitions, closures, or IPOs. Macro F1 is still our main metric. It calculates F1 for each of the four classes and takes a simple average, so the small classes count just as much as operating and the model can't look good by only getting the majority class right. Per class recall tells us what share of companies in each outcome the model actually finds, for example what percent of the companies that actually went public got labeled IPO. This week we also added per class precision, which is the share of the model's predictions for a class that are correct. We added it because of what happened with Random Forest V1 last week. It found 73% of IPO companies on validation, which looked good on its own, but it only found 39% of operating companies, meaning it was labeling way too many companies as one of the small classes. Recall alone doesn't show that, but low precision on a small class does. We still checked validation accuracy against the 81.7% baseline as a sanity check, but we didn't use it to pick the model.

Week 7 metrics (training and validation, all variations):

Metric	**V1 Train**	**V1 Validation**	**V2 Train**	**V2 Validation**	**V3 Train**	**V3 Validation**
Macro F1	0.388	0.365	0.668	**0.435**	0.976	0.412
Recall (Operating)	0.535	0.526	0.745	0.699	0.982	0.885
Recall (Acquired)	0.581	0.538	0.922	0.564	0.998	0.350
Recall (Closed)	0.559	0.491	0.896	0.433	1.000	0.193
Recall (IPO)	0.787	0.727	1.000	0.460	1.000	0.167

Precision (Operating)	0.947	0.937	0.988	0.919	1.000	0.866
Precision (Acquired)	0.257	0.236	0.476	0.291	0.940	0.331
Precision (Closed)	0.192	0.168	0.397	0.201	0.894	0.230
Precision (IPO)	0.132	0.123	0.474	0.207	1.000	0.327

As the model gets more complex, training performance climbs fast while validation performance peaks in the middle, which is the same overall pattern we saw with Random Forest. Training macro F1 goes from 0.388 (V1) to 0.668 (V2) to 0.976 (V3), while validation macro F1 goes 0.365, 0.435, 0.412, peaking at V2 and falling back at V3. The train minus validation gap in macro F1 grows from 0.023 (V1) to 0.233 (V2) to 0.564 (V3). Training was also greater than or equal to validation for every metric in every variation, which is what we'd expect.

V1 underfits. Its training and validation scores are almost the same (0.388 vs. 0.365), so it isn't memorizing anything, it's just too simple to separate four outcomes with depth 3 trees and slow learning steps. V3 has the opposite problem. With big learning steps and 300 rounds of depth 8 trees, it basically memorizes the training set (macro F1 0.976, recall at or above 0.982 for every class). On validation it falls back toward predicting operating (0.885 recall) and loses most of what it learned about the other classes, with recall dropping to 0.350 for acquired, 0.193 for closed, and 0.167 for IPO. That's a drop of 0.65 to 0.83 points between training and validation recall for the small classes. Since each round chases the leftover errors on specific training companies, by the later rounds the trees are fitting patterns that only exist in the training data.

V2 sits between the two. There is still clear overfitting, especially on the small classes (IPO recall falls from 1.000 on training to 0.460 on validation, and closed falls from 0.896 to 0.433), but it has the best validation macro F1 of the three and it's the only variation that keeps operating recall around 70% while still finding close to half of each minority class.

Looking only at validation, each variation wins on something different, and this is where adding precision helped the most. V1 has the highest validation recall for closed (0.491) and IPO (0.727), but that comes from guessing those labels too often, which shows up in its precision (0.168 for closed and 0.123 for IPO). V3 has the highest precision on all three small classes (0.331 acquired, 0.230 closed, 0.327 IPO) and the highest accuracy (77.1%), but only because it rarely predicts them at all, missing 83.3% of IPOs and 80.7% of closed companies. V3's accuracy being the closest to the 81.7% all operating baseline is really more of a warning sign than a good thing. V2 lands in between on almost every metric. It improves IPO precision from 0.123 to 0.207 over V1 and acquired precision from 0.236 to 0.291, while still catching 56.4% of acquired, 46.0% of IPO, and 43.3% of closed companies. None of the three variations beat the 81.7% baseline on accuracy (V1 52.8%, V2 66.2%, V3 77.1%), which we expected with balanced class weights and is the reason we grade on macro F1 instead.

In the end, we selected V2 (Moderate: learning_rate=0.10, max_depth=6, max_iter=200) as the winning model for the week. It has the highest validation macro F1 (0.435, vs. 0.365 for V1 and 0.412 for V3), which is the metric we chose specifically for this imbalanced, multi class problem. V1 is stable across training and validation but too weak to be useful, and V3's near perfect training scores don't hold up on validation. V2 also beats our Week 6 winner, Random Forest V2:

Model	**Val Macro F1**	**Recall (Operating)**	**Recall (Acquired)**	**Recall (Closed)**	**Recall (IPO)**	**Val Accuracy**

Week 6 Winner: Random Forest V2 (max_depth=15, n_estimators=200)	0.422	**0.724**	0.515	0.399	0.338	**0.673**
Week 7 Winner: Gradient Boosting V2 (learning_rate=0.10, max_depth=6, max_iter=200)	**0.435**	0.699	**0.564**	**0.433**	**0.460**	0.662

Compared to that forest, Gradient Boosting V2 improves validation recall on all three minority classes, with IPO going from 0.338 to 0.460, acquired from 0.515 to 0.564, and closed from 0.399 to 0.433, while only giving up a little operating recall (0.724 to 0.699). V2 still misses more than half of IPOs on validation and has a noticeable train validation gap, so it isn't a strong model in absolute terms, but it is the best model we've trained so far, and it's the one we're carrying forward (week7_gb_moderate.pkl) for test set evaluation.